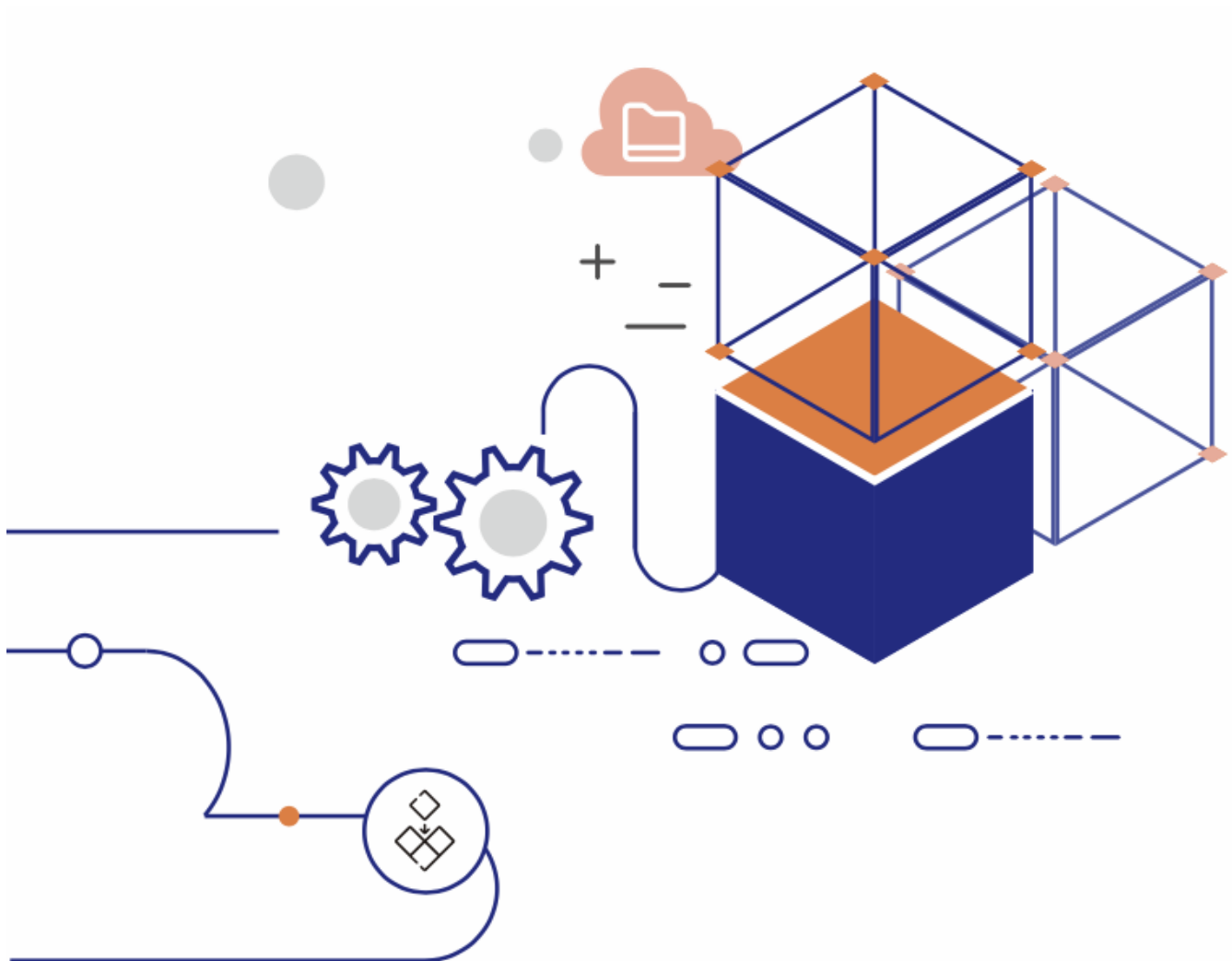


Table of Contents

- **EloqSQL**
 - EloqSQL: A Distributed High-Performance SQL Database for Elastic Performance at Any Scale
 - Benchmark Report of EloqSQL
- **EloqKV**
 - EloqKV: Blazing-Fast, Distributed, and Transactional — A Redis-Compatible KV Store Redefined
 - Benchmark Report of EloqKV
- **Appendix**
 - Assemble your database using Data Substrate



EloqSQL: A Distributed High-Performance SQL Database for Elastic Performance at Any Scale

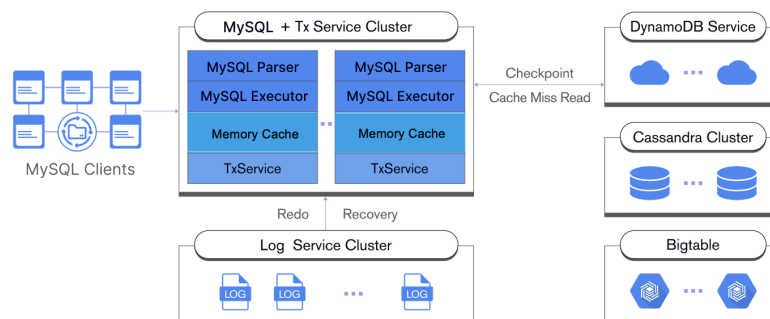
Introduction

In today's data-driven world, organizations face the challenge of managing ever-increasing volumes of data while ensuring high performance, scalability, and cost-effectiveness. Traditional database systems often struggle to meet these demands, leading to bottlenecks and performance limitations.

EloqSQL is a revolutionary distributed SQL database that addresses these challenges head-on. Powered by its innovative Data Substrate, EloqSQL delivers exceptional elasticity, scalability, and performance for latency-sensitive workloads, making it an ideal choice for modern enterprises.

Architecture

EloqSQL is a decoupled distributed database powered by Data Substrate. Its architecture includes a frontend compute engine compatible with the MySQL protocol. Within Data Substrate, the TxService is responsible for caching hot data and managing transaction processing, while the LogService handles data persistence. LogService replicas are distributed across different availability zones (AZs) to ensure tolerance to AZ-level failures. The underlying storage layer supports pluggable key-value (KV) storages, such as AWS DynamoDB, Google Bigtable, and Cassandra. These cloud storage services store cold data for cache misses and provide high availability for baseline data.



Key Features

MySQL Compatibility:

- Seamless integration with existing MySQL applications
- Leverages familiar MySQL protocol and syntax

Elastic Parallel Logging:

- Patented one-phase commit technique for distributed transaction performance
- 4x improvement in transactions per second in single node mode compared to MySQL
- Eliminates logging bottlenecks for write-intensive workloads

Elastic Memory Cache:

- Minimizes read latency with highly scalable in-memory data storage
- Supports hash and range partitioning for efficient data distribution
- Automatic scaling and rebalancing for optimal performance
- Cold data checkpointed to cloud storage for efficient cache miss read

Decoupled Cloud Storage:

- Independent scaling of storage and compute resources
- Cost-effective management of large datasets
- Optimized resource utilization for cold data
- Avoid cloud vendor lock-in and future-proof your data ecosystem with EloqSQL's seamless hybrid cloud storage

Use Cases

EloqSQL's unique blend of performance, elasticity, and cost-effectiveness makes it ideal for a wide range of use cases across industries. Here are just a few examples:

- **FinTech:** Payment Processing: Handle high-volume transactions with blazing-fast speed and rock-solid reliability. Guarantee data consistency while keeping the latency of complex transactions low.
- **Gaming:** Game Persistence: Deliver seamless gameplay experiences with reliable transaction handling and consistent game state management. Ensure players never lose progress or encounter disruptions.
- **E-Commerce:** Order Management: Process orders quickly and efficiently with elastic scalability that handles peak traffic without delays. Ensure data accuracy and prevent order fulfillment errors.
- **SaaS:** Metadata Management: Manage huge amount of metadata of your SaaS platform efficiently with EloqSQL's distributed architecture. Scale seamlessly to accommodate growing metadata volumes without compromising performance or availability.

EloqSQL: Embrace the Future of Distributed SQL:

Gone are the days of performance trade-offs and inflexible scalability in distributed SQL. EloqSQL rewrites the rules with:

- **Elastic Scaling on Demand:** Scale seamlessly to match your workload. Need blazing-fast queries? Scale up the compute engine. Facing write-heavy traffic? Scale out the log service. EloqSQL adapts precisely, ensuring optimal performance without overpaying.
- **Cost-Effective Efficiency:** Leave expensive two-phase commit behind. EloqSQL's innovative architecture delivers exceptional performance without unnecessary bloat, significantly reducing operational costs compared to traditional NewSQL systems.
- **Ditch the Clunky Workarounds:** Forget about complex sharding and cumbersome data management. EloqSQL simplifies your infrastructure, empowering you to focus on building great applications, not battling database overhead.

EloqSQL is the future of distributed SQL. Are you ready to break free?

Benchmark Report of EloqSQL

Introduction

The demand for high-performance, scalable databases that can meet the challenges of modern data-intensive applications continues to grow. NewSQL databases have emerged to address these needs, offering the transactional consistency of traditional RDBMS systems with the scalability of NoSQL databases.

While NewSQL promises both scalability and transactional consistency, many struggle with performance. Compared to single-node solutions like MySQL, their cost-efficiency suffers, and high latency renders them unsuitable for latency-sensitive tasks.

EloqSQL shatters NewSQL performance barriers with a unique approach: in-memory transaction processing powered by data substrate, a one-phase commit protocol to minimize disk I/O, and asynchronous key-value store access to eliminate latency bottlenecks.

To illuminate EloqSQL's edge, we'll conduct experiments focusing on:

- Benchmarking EloqSQL against leading NewSQL databases on mixed workload including distributed transaction to prove its performance superiority.
- Exposing the limits of traditional databases and underscores the crucial role of scalable memory for consistent performance, with AWS RDS plummeting due to cache misses, unfixable even by adding read replicas.
- Revealing how write-intensive workloads in the cloud benefit less from scaling CPU and memory, demonstrating the value of EloqSQL's decoupled architecture. Instead, focusing on scaling the true bottleneck like the log service unlocks unmatched performance.

Experiment I:

In the first scenario, we compare the performance of EloqSQL, a distributed SQL database powered by Data Substrate, with a popular open-source NewSQL database (refer to NewSQL-X in this report). The goal of this comparison is to evaluate the performance of EloqSQL under various workloads especially distributed transaction workloads and its potential to deliver exceptional performance for demanding applications.

A mixed workload was used, simulating a combination of read and write operations to assess overall performance.

- Distributed Transaction: Transactions spanned multiple database instances, ensuring a thorough evaluation of distributed transaction handling capabilities. Each transaction involved a combination of update, delete, or insert queries, simulating the complexity of real-world applications.
- Single Update: Transactions featured a single non-index update, assessing the efficiency of handling basic write operations.
- Point Select: Transactions involved a single point select query, measuring the speed and efficiency of basic read operations.

Hardware and Software:

To test against NewSQL database in the same hardware configuration, we deploy EloqSQL in co-locate mode, i.e. deploy TxService, LogService and KVStore in the same node. The deployment details is as follows:

Service type	Node type	Node count	Disk count
NewSQL-X	n2-standard-32	3	500G*1
Service type	Node type	Node count	Disk count
EloqSQL	n2-standard-32	3	500G*1

To illustrate EloqSQL's parallel logging capabilities and maximize I/O performance, we setup a new EloqSQL deployment with three 50GB SSD disks. Note that three 50GB SSD disks are significantly cheaper than 96 cores of compute resources in the cloud.

Service type	Node type	Node count	Disk count
EloqSQL	n2-standard-32	3	350G*1 + 50G*3

Disk Considerations:

- NewSQL-X's official benchmark report employed local SSDs, which cannot persist data after instance restarts.
- To align with cloud-native environments and ensure data persistence, this benchmark utilized PD-SSD disks in GCP for both databases.

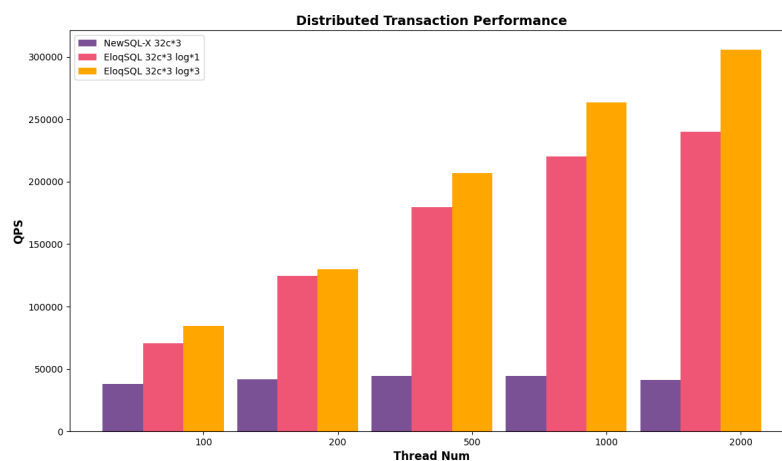
Results

Firstly, we study the throughput of the two databases under different workloads.

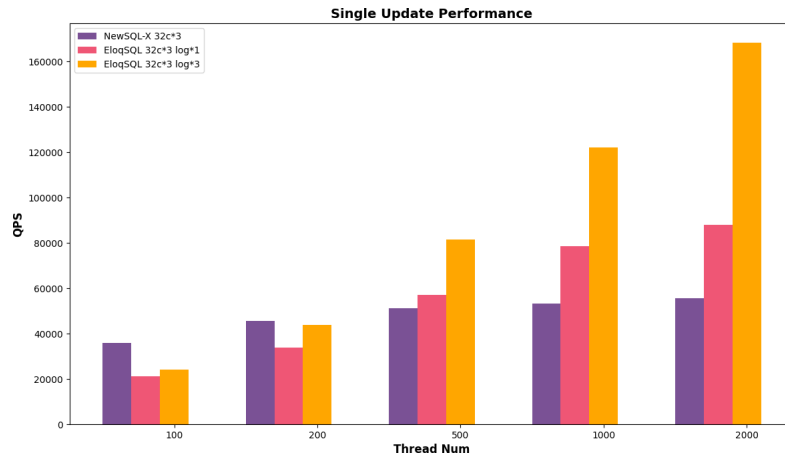
X-axis: Represents the varying thread numbers employed during the benchmark, simulating different levels of concurrent database access.

Y-axis: Measures the QPS (Queries Per Second).

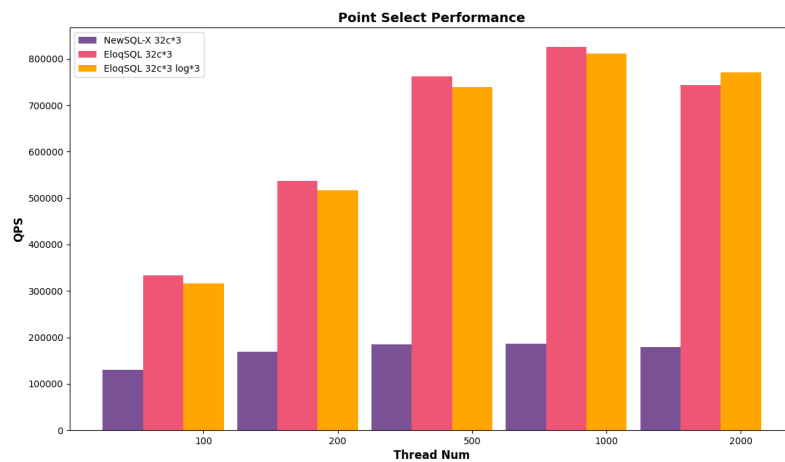
- Distributed Transaction Workload:



- Single Update Workload:



- Point Select Workload:

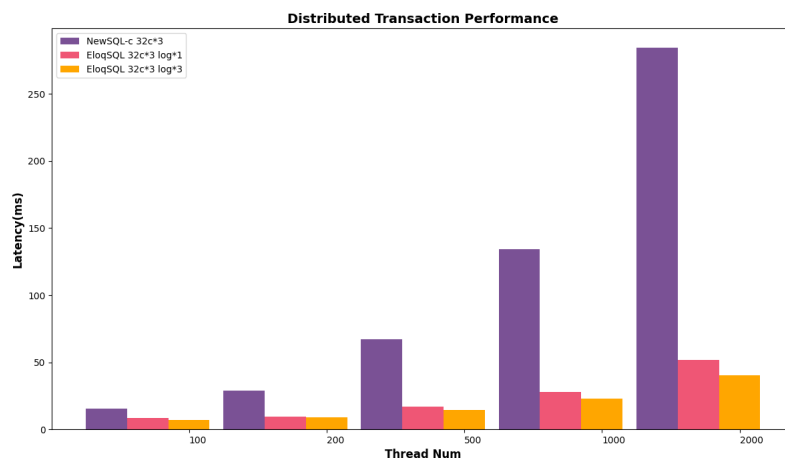


Next, we study the latency of the two databases.

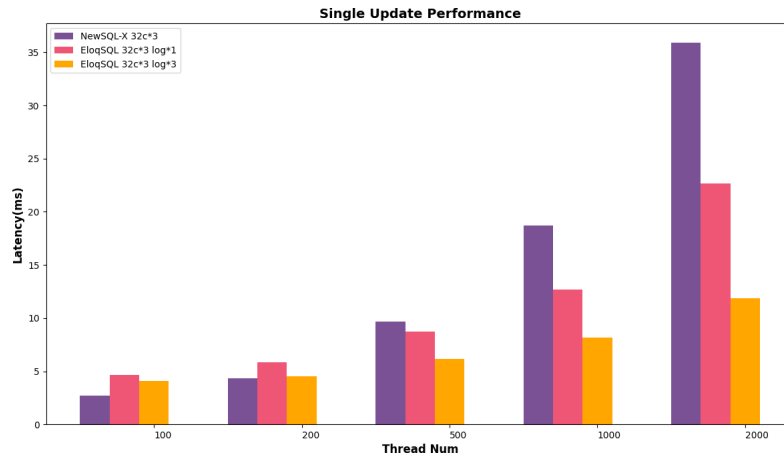
X-axis: Represents the varying thread numbers employed during the benchmark, simulating different levels of concurrent database access.

Y-axis: Measures the Latency.

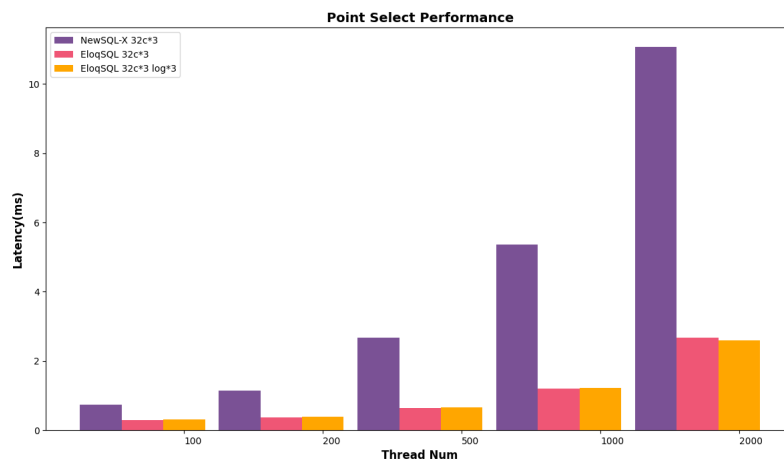
- Distributed Transaction Workload:



- Single Update Workload:



- Point Select Workload:



The above results shows that EloqSQL consistently outperformed NewSQL-X in terms of QPS and latency among different kinds of workloads. This advantage is particularly evident for distributed transactions. When configured with three additional disks for Parallel Logging, EloqSQL further elevates its write throughput.

Key Takeaways

Compared with NewSQL, EloqSQL has superior ability to handle distributed transactions and deliver high performance under demanding workloads. Its innovative Data Substrate architecture enables it to achieve significantly higher QPS compared to NewSQL-X, making it a compelling choice for organizations seeking a high-performance, scalable NewSQL database solution.

Expeiment II:

Many organizations fall into the trap of adding read replicas to RDS MySQL, hoping to address cache misses. This experiment reveals the futility of such efforts and introduces EloqSQL's memory scale out capability for maintaining performance under memory constraints.

Hardware and Software:

To ensure a level hardware playing field for benchmarking against MySQL RDS, we deployed EloqSQL in co-locate mode, housing TxService, LogService, and KVStore within the same node. Here's a breakdown of the deployment configurations:

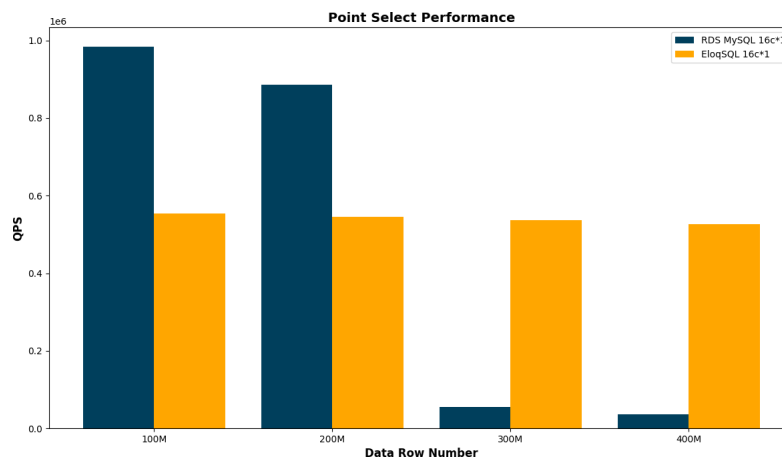
- EloqSQL: 6 nodes with 16 cores/64GB memory each node
- AWS RDS(MySQL): 1 read-write node + 5 read-only nodes with 16 cores/64GB memory each node

Results

X-axis: Represents the varying hot data size from 100 million records to 400 million records, simulating the cache miss as data size increases.

Y-axis: Measures the QPS (Queries Per Second).

- We utilize Point Select with uniform distribution to randomly select search keys, ensuring each key has an equal probability of being chosen.



Results demonstrate that AWS RDS outperforms EloqSQL when data size is small and can be fully cached in a single node's memory. As data size increases, the QPS of AWS RDS drops sharply which is caused by a large amount of cache miss. However, EloqSQL can keep consistent performance.

Key Takeaways

- For datasets that fit within a single node's memory (under 200 million records), read replicas provide a performance advantage over distributed memory architectures. This is due to: RDS read replicas operate as independent databases, ensuring all queries are executed locally, eliminating the overhead of remote communication. Conversely, EloqSQL's distributed memory architecture necessitates remote memory access via RPC (Remote Procedure Calls) for read requests spanning multiple shards, introducing latency and potentially impacting performance.
- For datasets exceeding single-node memory capacity (around 300 million records), distributed memory architectures like EloqSQL outperform read replicas in read performance. The reason is twofold. 1. RDS cache misses hamper performance: as data surpasses single-node memory, RDS experiences frequent cache misses due to randomly distributed queries, leading to significantly slower reads. Even adding more read replicas doesn't mitigate this issue. 2. EloqSQL scales to cache everything: EloqSQL's distributed memory architecture shines in these scenarios. It scales horizontally, effectively caching all hot data across multiple nodes, ensuring consistent and high read performance regardless of dataset size.

Experiment III:

Harnessing its decoupled architecture, EloqSQL empowers strategic resource allocation to optimize performance across different components. In this benchmark, we reveals the fact that scaling CPU and memory doesn't help when disk becomes the bottleneck for write-intensive workloads. We demonstrate the value of EloqSQL's decoupled architecture which supports to focus on scaling the true bottleneck like the log service by allocating additional disks to unlocks unmatched performance.

Hardware and Software:

Leveraging EloqSQL's decoupled architecture, we strategically distributed its components across nodes for optimal resource utilization. Our baseline configuration featured a 48-core TxService node coupled with a single-disk LogService node. We then explored performance gains by scaling the TxService node to 64 cores and increasing LogService disk count to 3.

Service type	Node type	Node count	Disk count
txservice	n2-standard-48	1	1
logservice	n2-standard-8	1	1
cassandra	n2-standard-8	1	1
Service type	Node type	Node count	Disk count
txservice	n2-standard-64	1	1
logservice	n2-standard-8	1	1
cassandra	n2-standard-8	1	1
Service type	Node type	Node count	Disk count
txservice	n2-standard-48	1	1
logservice	n2-standard-8	1	3
cassandra	n2-standard-8	1	1

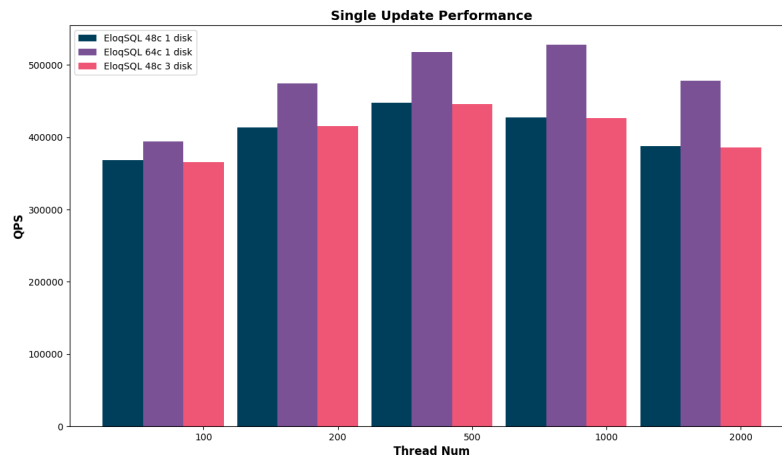
Results

Firstly, we study the read throughput of different choice of scaling.

X-axis: Represents the varying thread numbers employed during the benchmark, simulating different levels of concurrent database access.

Y-axis: Measures the QPS (Queries Per Second).

- Point Select Workload



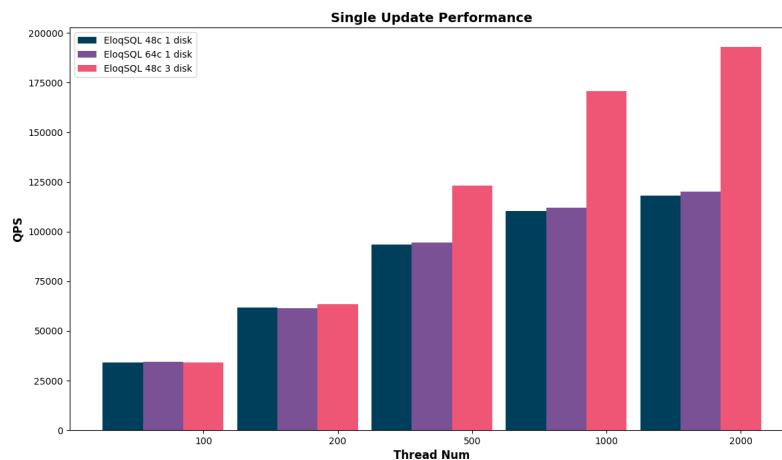
Results demonstrate that scaling CPU cores from 48 to 64 yields a notable increase in read throughput, while scaling log resources offers no improvement. This aligns with expectations, as read-intensive workloads primarily benefit from additional processing power rather than enhanced logging capabilities.

Next, we study the write throughput of different choice of scaling.

X-axis: Represents the varying thread numbers employed during the benchmark, simulating different levels of concurrent database access.

Y-axis: Measures the QPS (Queries Per Second).

- Single Update Workload



Results show that scaling CPU beyond 48 cores didn't enhance write throughput, but scaling logs did, uncovering disk I/O as the system's primary bottleneck.

Key Takeaways

EloqSQL's revolutionary decoupled architecture breaks free from monolithic scaling limitations. Its distinct components—CPU/memory, logs, and key-value store—scale independently, enabling targeted resource allocation. Optimize write throughput by scaling logs, unleash read performance with CPU/memory scaling, and accommodate massive datasets with key-value store expansion.

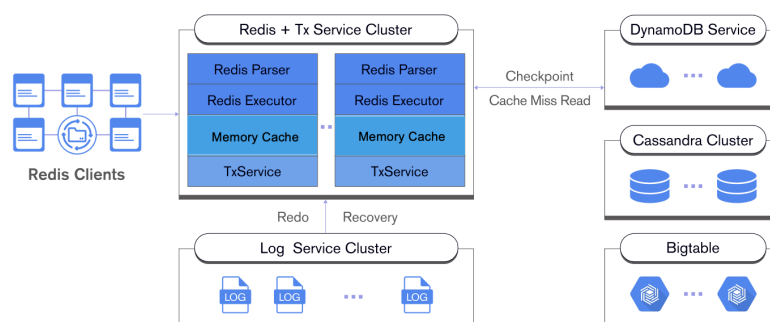
EloqKV: Blazing-Fast, Distributed, and Transactional — A Redis-Compatible KV Store Redefined

Introduction

In today's fast-paced digital landscape, speed and reliability are paramount. Latency is the enemy, and data integrity is a non-negotiable. Traditional caching solutions like Redis offer fast reads, but lack the robust transaction support needed for mission-critical applications. Introducing EloqKV, a revolutionary distributed transactional cache powered by Data Substrate, that shatters performance barriers while ensuring rock-solid data consistency.

Architecture

EloqKV is a decoupled distributed database powered by Data Substrate. Its architecture includes a frontend compute engine compatible with the Redis protocol. Within Data Substrate, the TxService is responsible for caching hot data and managing transaction processing, while the LogService handles data persistence. LogService replicas are distributed across different availability zones (AZs) to ensure tolerance to AZ-level failures. The underlying storage layer supports pluggable key-value (KV) storages, such as embedded RocksDB, AWS DynamoDB, Google Bigtable, and Cassandra. These storage services store cold data for cache misses and provide high availability for baseline data.



Beyond Caching, Embracing Transactions

Unlike its peers, EloqKV transcends the limitations of simple key-value stores. It seamlessly integrates full ACID (Atomicity, Consistency, Isolation, Durability) properties across distributed transactions and clusters. This unlocks unprecedented functionality, empowering you to:

- **Ditch the Duo:** Say goodbye to the cumbersome MySQL + Redis combo. EloqKV eliminates cache coherence issues entirely, simplifying your architecture and boosting efficiency.
- **Transactional Confidence:** Ensure data integrity across reads and writes, even in complex distributed environments.
- **Unlock New Application Scenarios:** Tackle use cases beyond traditional caching, venturing into the realm of transactional microservices and stateful data management.

Cost-Conscious Performance Made Simple

EloqKV leverages Data Substrate's innovative architecture to deliver performance and cost-effectiveness in perfect harmony:

- **Memory for Speed:** Frequently accessed data dances in-memory, guaranteeing lightning-fast reads and blazing-fast write performance through parallel logging.
- **Cloud for Cold Data:** As data cools, it gracefully migrates to cost-effective cloud key-value stores, freeing up precious DRAM resources.
- **Asynchronous Checkpoints:** Minimize IOPS requirements and optimize performance while keeping transactional reads readily available for cache misses.
- **Operational Efficiency:** Slash operational costs with cloud storage and enjoy streamlined maintenance thanks to Data Substrate's modular design.

Leverage Parallel Processing Power:

EloqKV doesn't settle for single-threaded limitations. It flexes its multi-threaded architecture to:

- **Maximize Hardware Potential:** Uncork the full power of modern CPUs, maximizing resource utilization and delivering exceptional performance.
- **Crush Bottlenecks with Concurrent Execution:** Say goodbye to single-threaded limitations. EloqKV's multi-threaded architecture handles tasks simultaneously, significantly boosting performance and efficiency.

Scale on Demand, Optimize on the Fly

EloqKV adapts to your dynamic needs, scaling seamlessly to match your workload:

- **Memory Scaling:** When hot data demands grow, instantly expand in-memory capacity for uninterrupted performance.
- **Log Service Optimization:** Handle surges in write traffic by effortlessly scaling the log service.
- **Cloud Storage Growth:** As historical data accumulates, seamlessly expand the cloud storage layer to accommodate your evolving needs.

Use Cases

- **GenAI-Ready File System:** Empower the era of general artificial intelligence with a metadata engine capable of handling billions of files seamlessly.
- **Gaming:** Leaderboards and Player Profiles: Manage global leaderboards and player data with high concurrency and low latency, ensuring a smooth and responsive experience for millions of gamers.
- **Ad-tech:** Real-time Bidding and Ad Serving: Handle high-volume ad requests and deliver relevant ads with millisecond latency, maximizing ad performance and revenue.
- **E-commerce:** Inventory Management and Product Catalog: Ensure real-time inventory updates and fast product searches for millions of items, optimizing customer experience and preventing stockouts.

Benchmark Report of EloqKV

Experiment I EloqKV as Memory Cache:

In the first senario, we evaluate EloqKV as an in-memory cache, focusing on its single-node performance. We compare EloqKV with Redis, a widely used in-memory data store, and DragonflyDB, a newer option boasting high performance due to its multi-threaded architecture and optimized implementation leveraging modern innovations.

Hardware and Software:

Server Machine:

Service	Node type	Node count
Redis 7.2.5	c7g.8xlarge	1
DragonflyDB 1.21.2	c7g.8xlarge	1
EloqKV 0.7.4	c7g.8xlarge	1
Client - Memtier	c6gn.8xlarge	1

We follow the official instructions to setup [EloqKV](#), [DragonflyDB](#) and [Redis](#).

For Redis, we disable AOF and RDB persistency.

```
redis-server --save "" --appendonly no
```

For DragonflyDB, we only need to disable checkpointing since it does not yet support logging.

```
dragonfly --dbfilename=
```

For **EloqKV**, we disable persistent storage and turn off WAL (Write-Ahead Logging) in its `config.ini` file.

```
# set it to off to turn off persistent storage
enable_data_store=off
# set it to off to turn off WAL
enable_wal=off
```

Write-Only Workload

To evaluate **EloqKV**'s write performance, we run `memtier_benchmark` with ratio of 1:0 (write-only) with the following configuration:

```
memtier_benchmark -t $thread_num -c $client_num -s $server_ip -p
$server_port --distinct-client-seed --ratio=1:0 --key-prefix="kv_" --key-
minimum=1 --key-maximum=5000000 --random-data --data-size=128 --hide-
histogram --test-time=300
```

- **-t**: Number of threads, which was set to a fixed value of 32.
- **-c**: Number of clients per thread. We configured it to 4, 8, 16 and 32 to evaluate different concurrency levels. This resulted in total concurrency of 128, 256, 512 and 1024, calculated as $\text{thread_num} \times \text{client_num}$.
- **--ratio**: Set:Get ratio, 1:0 for write-only workload.

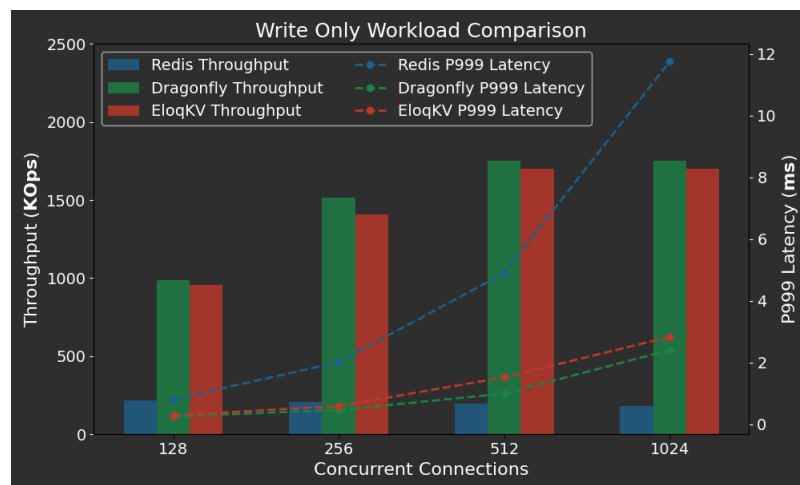
Results

Below are the results of the write-only workload.

X-axis: Represents the varying concurrencies ($\text{thread_num} \times \text{client_num}$), simulating different levels of concurrent database access.

Left Y-axis: Throughput in QPS (Queries Per Second).

Right Y-axis: 99.9 Percentile latency in milli seconds (ms).



EloqKV and DragonflyDB both outperform Redis due to their support for multiple worker threads. **EloqKV** delivers almost the same high throughput and low latency as DragonflyDB across various concurrency scenarios.

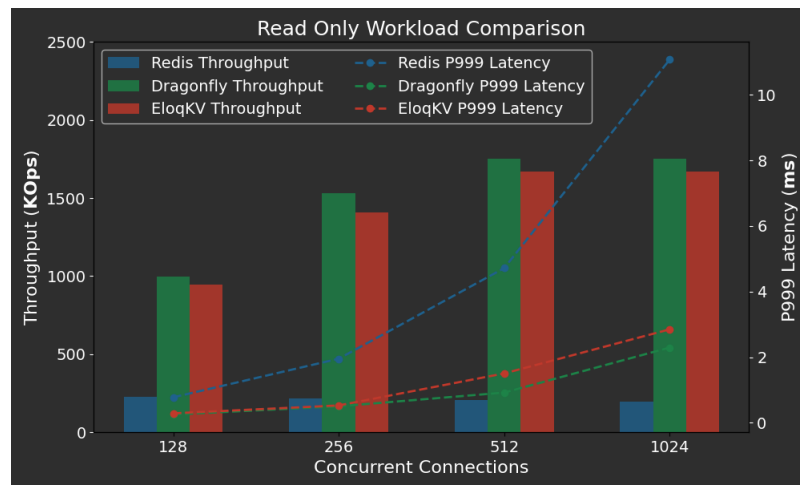
Read-Only Workload

For the read-only workload, we adjusted the Set:Get ratio to 0:1 (read-only):

```
memtier_benchmark -t $thread_num -c $client_num -s $server_ip -p
$server_port --distinct-client-seed --ratio=0:1 --key-prefix="kv_" --key-
minimum=1 --key-maximum=5000000 --random-data --data-size=128 --hide-
histogram --test-time=300
```

- `--ratio`: Set to 0:1 for read-only operations.

Results



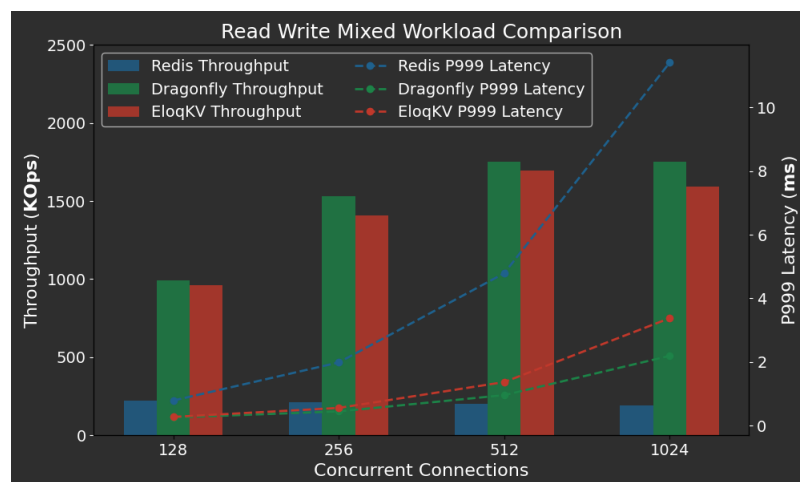
Again, **EloqKV** offers slightly lower throughput, and slightly higher but still very respectable latency compared to DragonflyDB. Both **EloqKV** and DragonflyDB significantly outperform Redis, both in throughput and in latency.

Mixed Write-Read Workload

Finally, the mixed workload with a 1:10 ratio of Put:Get:

```
memtier_benchmark -t $thread_num -c $client_num -s $server_ip -p
$server_port --distinct-client-seed --ratio=1:10 --key-prefix="kv_" --key-
minimum=1 --key-maximum=5000000 --random-data --data-size=128 --hide-
histogram --test-time=300
```

- `--ratio`: Set to 1:10 for mixed write-read operations.



EloqKV exhibits similar throughput to DragonflyDB. As concurrency increases, **EloqKV** shows a slightly higher P999 latency than DragonflyDB, but remains under 4ms even with over a thousand concurrent connections.

Key Takeaways

Based on the previous experiments, we can obtain some interesting observations. In this section, we will give some (opinionated) analysis.

Single Worker vs Multiple Workers

EloqKV and DragonflyDB can significantly outperform *single process* Redis on a modern multi-core server. The difference is the result of a fundamental [design philosophy](#) took by Redis. Redis restricts internal in-memory data structure operations to a single worker thread, while multiple IO threads handle networking and persistency. This choice, though greatly simplifies Redis's design, naturally limits performance on multi-core systems. In comparison both **EloqKV** and DragonflyDB allow multiple workers.

Do We Really Need to Specialize?

Compared with DragonflyDB, **EloqKV** currently lacks a few optimizations such as [io_uring](#) based networking. Due to these limitations, our profiling shows that **EloqKV** is bounded by the networking stack when serving the workloads in the experiments.

Even so, as the experiments demonstrated, **EloqKV** works *almost* as well as DragonflyDB on a workload that DragonflyDB was specifically designed and optimized for. This begs the question of whether designing special database software for limited use cases is profitable. Unlike Redis and DragonflyDB, **EloqKV** is much more than a single node memory cache. It excels in clustered, durable, and fully ACID-compliant transactional setups.

Experiment II EloqKV as Durable KV Store:

In this experiment, we compare **EloqKV** with Apache Kvrocks, a Redis-compatible NoSQL database that supports persistence. We evaluate the performance of **EloqKV** and Kvrocks under write-intensive and mixed workloads. To ensure data durability, we enable fsync Write-Ahead Logging (WAL) for both databases. For **EloqKV**, both the transaction service and log service are deployed on the same node (c7gi.8xlarge). To fully utilize available disk IO, we start two LogService processes to write WAL logs in **EloqKV**.

Hardware and Software:

Server Machine:

Service type	Node type	Node count	Local SSD	EBS gp3 volume
Kvrocks	c7gd.8xlarge	1	1 x 1900GB NVME	1
EloqKV	c7gd.8xlarge	1	1 x 1900GB NVME	1

For **EloqKV**, we enable persistent storage and turn on WAL (Write-Ahead Logging).

```
# set it to on to turn on persistent storage
enable_data_store=on
# set it to on to turn on WAL
enable_wal=on
```


For Kvrocks, we mainly changed two configuration options.

```
# If yes, the write will be flushed from the operating system
# buffer cache before the write is considered complete.
# If this flag is enabled, writes will be slower.
# If this flag is disabled, and the machine crashes, some recent
# writes may be lost. Note that if it is just the process that
# crashes (i.e., the machine does not reboot), no writes will be
# lost even if sync==false.
#
# Default: no
# rocksdb.write_options.sync no
rocksdb.write_options.sync yes

# The number of worker's threads, increase or decrease would affect the
# performance.
# workers 8
workers 24
```

Workload

Disk performance plays a critical role in write-intensive workloads. Therefore, we conducted benchmarks using both local SSDs and Elastic Block Store (EBS). Local SSDs offer low latency and high IOPS, making them ideal for high-performance needs. However, in a cloud setup data on local SSD may get lost if the virtual machine (VM) is stopped. On the other hand, EBS provides high availability, allowing the volume to be attached to a new VM if the original VM fails. Moreover, EBS is elastic, allowing precise control over number of volumes and their sizes. In our case, a 50GB [EBS gp3](#) volume is plenty for our WAL needs. Such a volume only cost \$4 per month while providing 3000 IOPS and 125 MB/s throughput. Given the distinct advantages and limitations of local SSDs and EBS, we conducted our experiments with both.

We run `memtier_benchmark` with the following configuration:

```
memtier_benchmark -t $thread_num -c $client_num -s $server_ip -p
$server_port --distinct-client-seed --ratio=$ratio --key-prefix="kv_" --
key-minimum=1 --key-maximum=5000000 --random-data --data-size=128 --hide-
histogram --test-time=300
```

- `-t`: Number of threads for parallel execution, which we set to 80.
- `-c`: Number of clients per thread. We set it to 5, 10, 20, 40 to evaluate different concurrency levels. This resulted in total concurrency values of 400, 800, 1600, and 3200, calculated as `thread_num × client_num`.
- `--ratio`: Set:Get ratio is set to 1:0 for write-only workload, and 1:10 for mixed workload.

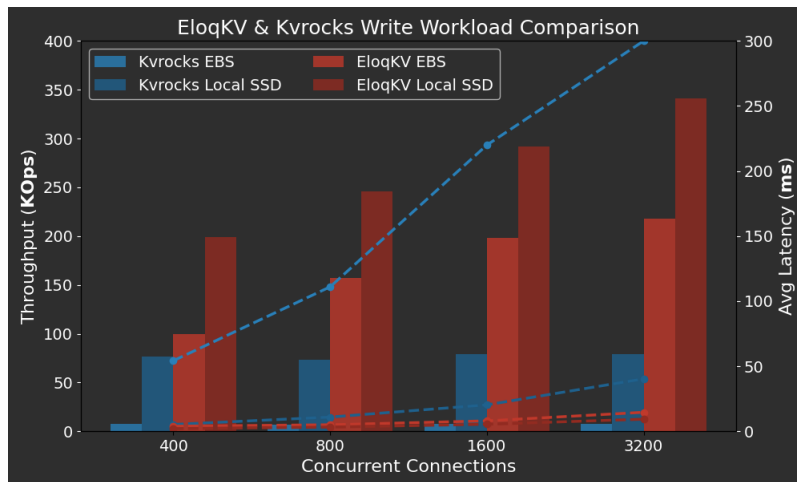
Results

Below are the results of the write-only workload.

X-axis: Represents the varying concurrencies ($\text{thread_num} \times \text{client_num}$), simulating different levels of concurrent database access.

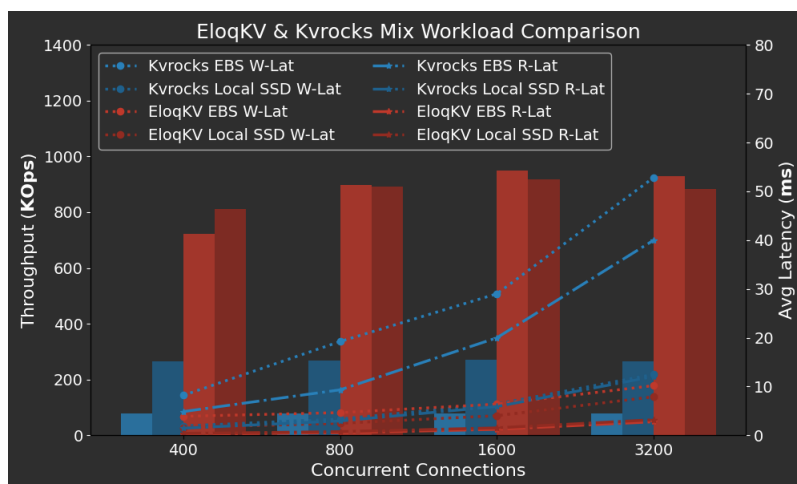
Left Y-axis: Throughput in Thousand OPS (Operations Per Second).

Right Y-axis: Average latency in milli seconds (ms).



EloqKV significantly outperforms Kvrocks on both EBS and local SSD. On EBS, **EloqKV** achieves a write throughput that is 10 times higher than Kvrocks, while on local SSD, it is 2-4 times faster. This performance improvement is due to **EloqKV**'s architecture, which decouples transaction and log services, allowing multiple log workers to write Write-Ahead Logs (WAL) and perform fsync operations in parallel, thereby enhancing overall throughput. Additionally, **EloqKV** maintains significantly lower latency compared to Kvrocks, even under high concurrency.

Below are the results of the mixed workload.



Results show that **EloqKV** outperforms Kvrocks on mixed workloads as well. **EloqKV** maintains a read latency of less than 1 ms even under a heavy mixed workload with nearly 900K OPS. In contrast, Kvrocks on EBS exhibits significantly higher latencies, with both read and write latencies exceeding 10 ms even at relatively low concurrency, and rising to over 50 ms as concurrency increases. Even on local SSDs, Kvrocks' read latency remains much higher than that of **EloqKV**. This demonstrates that **EloqKV** can sustain low read latency even when the cluster is under a heavy write workload.

Key Takeaways

In this experiment, we evaluate EloqKV and show its performance when data durability is strongly enforced. On a plain low end server, EloqKV can easily sustain over 200,000 writes per second with acceptable latency. While this is lower than the pure in-memory cache performance, it is still quite suitable for many real-world applications. Notice that this performance number is not much different from what a single-process Redis server can achieve on the same hardware in pure memory mode.

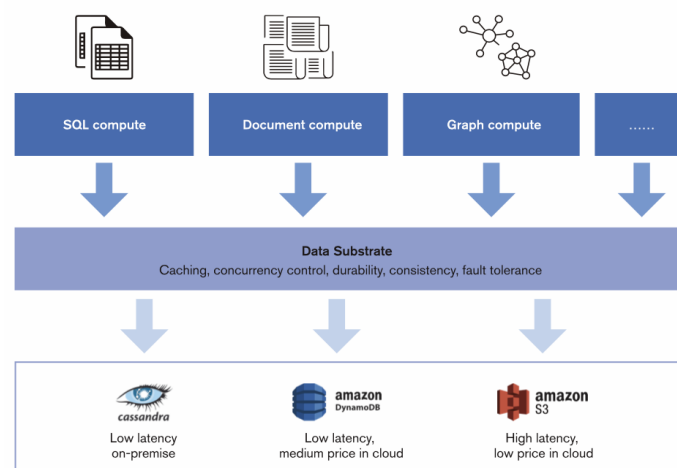
Appendix: Assemble your database using Data Substrate

In today's data-driven world, organizations face mounting challenges managing and utilizing information effectively. Traditional database systems often struggle to adapt to rapidly evolving business needs, leading to complex setups, rigid scalability, and operational headaches. Introducing Data Substrate, a revolutionary abstraction layer that empowers you to build customized, dynamic databases tailored to your specific requirements.

What is Data Substrate?

Think of Data Substrate as the invisible backbone of your data management solution. It acts as a unified platform, encapsulating critical functionalities commonly needed across diverse data scenarios. From ensuring consistency and data durability to handling concurrency and fault tolerance, Data Substrate takes care of the essential groundwork, freeing you to focus on building the bespoke database your business needs.

Moreover, Data Substrate's modular design and vertical scalability let you tailor it to your specific requirements. This means you can independently scale each layer – compute engines, Data Substrate itself, and cloud storage – to perfectly match your workload demands. Additionally, its cloud-powered cost-efficiency utilizes low-cost storage options for cold data, keeping your resources optimized. With a range of flexible engines at your disposal, you can craft a database perfectly suited to your application, whether it's a high-performance SQL engine for online transaction processing or a kv-based engine for flexible content management, and even wasm based engine for any user defined function.



Key Components:

- **Compute Engines:** The architecture's top layer consists of a variety of adaptable compute engines, including SQL, KV, document, and graph engines. These engines seamlessly integrate with Data Substrate, offering flexibility in data processing and analysis.
- **Data Substrate:** This core layer acts as the backbone of the architecture, providing essential functionalities:

- Caching: Optimizes performance by storing frequently accessed data in memory for rapid retrieval.
- Concurrency Control: Ensures transaction ACID and supports multi-write architecture.
- Data Persistence: Guarantees data durability by storing it persistently, even in case of system failures.
- Consistency: Maintains data coherence across multiple components and operations.
- Fault Tolerance: Enhances resilience by handling errors and fast recovery without data loss.
- Cloud Storage: Data Substrate integrates with diverse cloud storage solutions like AWS DynamoDB and Google Bigtable, serving two crucial purposes:
 - Cold Data Storage: Cost-effectively stores less frequently accessed data, reducing compute resource requirements.
 - Cache Miss Handling: Fetches data from cloud storage when it's not found in the cache, ensuring comprehensive data accessibility.
 - Break free from vendor lock-in and embrace true cloud independence with Data Substrate's seamless hybrid cloud storage architecture.

Why Choose Data Substrate?

Modern enterprises require nimble data systems that can seamlessly adapt to their unique workload demands. Data Substrate empowers you to break free from the limitations of traditional database approaches, addressing common pain points such as:

- Long, cumbersome data pipelines: Data Substrate streamlines data flow, eliminating redundant processing and simplifying your data infrastructure.
- Repetitive hand-coding of core functionalities: No more reinventing the wheel! Data Substrate provides a robust foundation, letting you focus on your specific data logic.
- Low resource utilization: Data Substrate ensures efficient resource allocation, scaling the right components to match your workload demands and preventing wasted capacity.
- Inconsistent data synchronization: Eliminate data siloes and ensure seamless data consistency across your entire system with Data Substrate's transactional cache mechanism.
- Choose your cloud, your way: Data Substrate empowers you to orchestrate your data seamlessly across multiple clouds, creating a hybrid environment that aligns perfectly with your business needs and empowers strategic decision-making.

Key Features of Data Substrate:

Data Substrate shines through its unparalleled elasticity and adaptability. It automatically adjusts to your needs, ensuring optimal performance regardless of workload variations. Here are some key highlights:

- Elastic Scaling: Adapt to diverse workloads effortlessly. For read-intensive scenarios, Data Substrate scales out its memory, enabling distributed caching and lightning-fast data retrieval.
- Parallel Write Optimization: No more bottlenecks! Data Substrate's patented one-phase commit protocol and parallel logging capabilities handle write-heavy workloads with ease, guaranteeing data durability and high availability even under extreme pressure.
- Cost-Effective Scalability: Large datasets are no match for Data Substrate. Seamlessly scale out your cloud storage without overloading compute resources, minimizing costs for infrequently accessed data.